



TCG: an approach to improving test-driven development by exploiting use case specifications

Thanh-Binh Trinh¹ · Ninh-Thuan Truong²

Received: 4 March 2025 / Accepted: 9 October 2025 / Published online: 24 October 2025

© The Author(s) under exclusive licence to The Society for Reliability Engineering, Quality and Operations Management (SREQOM), India and The Division of Operation and Maintenance, Lulea University of Technology, Sweden 2025

Abstract

Test-Driven Development (TDD) is a powerful software development methodology where tests are written before code to improve reliability, enable faster feedback, and simplify maintenance. However, TDD is time-consuming and requires experienced developers to write test cases. To address these challenges, this paper proposes an approach to improve TDD through automated test case generation from requirements. We implemented this approach as *TCG*, a tool that converts Cockburn-style use cases into test cases. The paper details the *TCG* tool's architecture and its algorithms for building CFGs and generating optimized test cases. By automating this process, we aim to reduce the required manual effort and enhance the coverage and quality of test cases. Additionally, the tool provides a user-friendly interface for uploading use case documents and displaying the generated test cases, making it accessible for software developers and testers. The approach is evaluated through several software projects, with initial results indicating that the generated test cases are minimal yet achieve high coverage, making them suitable for use in software development practice. These findings highlight the advantages of our approach. *TCG* enhances test coverage, reduces manual effort, and improves fault detection, making it suitable for integration into TDD workflows and practical software engineering environments.

Keywords Test-driven development · Control flow graphs · Test case generator · Use case specification

1 Introduction

Software development methodologies, from Waterfall to Agile frameworks like Scrum and Test-Driven Development (TDD), address diverse project demands. Beck (2002), Beck et al. (2001). Each methodology provides distinct advantages and is suited to specific types of projects. Among these, TDD stands out for its emphasis on early testing and iterative refinement. However, its adoption remains challenging due to the labor-intensive process of manually writing test cases—a task exacerbated during requirement volatility. This process is time-consuming, especially during the early stages of development when requirements are still evolving. Consequently, teams often compromise on TDD rigor, resulting in poor test coverage and misalignment between specifications and implementation. Automating test case generation directly from requirements would address this challenge and increase the practicality and scalability of TDD in real-world projects.

To understand how automated test generation improves TDD, it is essential to briefly revisit the TDD process. This development process (Astels 2003) follows a repetitive and incremental cycle: (1) writing a test from requirements, (2) coding to pass the test, and (3) refactoring for clarity. By enforcing continuous testing and immediate feedback, this approach not only supports early defect detection but also encourages clean, modular design. As a result, TDD has been shown to lead to higher-quality software in terms of both correctness and maintainability (Janzen and Saiedian 2005).

Consequently, writing effective test cases becomes a time-consuming and costly task during each testing cycle (Erdogmus et al. 2005). To address this issue, several

✉ Thanh-Binh Trinh
binh.trinhthanh@phenikaa-uni.edu.vn
Ninh-Thuan Truong
thuantrn@vnu.edu.vn

¹ Faculty of Information Systems, Phenikaa University, Hanoi, Vietnam

² VNU University of Engineering and Technology, VNU, Hanoi, Vietnam

automated test case generation methods have been proposed to save time, cost, and effort while providing more optimal and high-coverage test cases (Alagarsamy et al. 2023; Alrawashdeh et al. 2019; Schafer et al. 2024; Takerngsaksiri et al. 2024). These methods for automatically generating test cases can be classified into two types: code-based test case generation and specification-based test case generation.

Source code-based test case generation (Alagarsamy et al. 2023; Schafer et al. 2024), also known as white-box or structural testing, involves creating test cases based on the internal structure and implementation of the code. This approach aims to ensure that the code behaves as expected, and all paths, branches, and conditions are tested. However, source code-based test case generation requires existing code as input to generate test cases. This approach directly contradicts the TDD methodology, where tests drive the development process and guide the writing of the code from the outset. Therefore, source code-based test generation undermines TDD by breaking the link between tests and initial requirements.

Specification-based test case generation (Alrawashdeh et al. 2019; Takerngsaksiri et al. 2024) is a method of creating tests for software applications based on the specifications or requirements of the system rather than the existing code. By focusing on the requirements, specification-based test case generation ensures that all necessary functionalities are tested early in the development process, leading to more robust and reliable software. Furthermore, it helps maintain alignment between the tests and the initial requirements, reinforcing the TDD practice of driving development through testing.

On the other hand, specification-based testing offers several advantages over code-based testing. It allows specifications to guide the design of software tests, reducing both the cost of testing and overall software development expenses. This approach also helps developers identify issues within the specifications early on. Furthermore, specification-based testing saves time and resources by enabling testing to occur at an earlier stage in the development life cycle (Ribeiro et al. 2018). Thus, many researchers focus on generating test cases from software specifications (Khurana et al. 2016; Ribeiro et al. 2018; Sahoo et al. 2017; Teixeira and Silva 2018). However, there is a noticeable gap in research regarding the generation of test cases from a formalized model of use cases during the early requirements phases. These studies often lack the application of acceptable software test coverage criteria to measure the adequacy and reliability of the test cases produced. Additionally, there is a need for a fully automated and detailed approach to test case generation. Moreover, integrating these methodologies into the TDD process could enhance the overall quality and effectiveness of the software development lifecycle. To bridge this gap, further research is needed to develop methodologies that:

- Automatically generating test cases from a formalized model of use cases typically involves leveraging tools and techniques in specification-based testing, which uses formal models of system requirements to generate test cases.
- Apply accepted software test coverage criteria to ensure the adequacy and reliability of test cases.
- Provide a fully automated approach for detailed test case generation.

In this paper, we propose an approach to enhance TDD through automated test case generation from formalized use case specifications. In short, the main contributions of our approach can be summarized as follows: (1) An algorithm to convert use case specifications in Cockburn format into CFGs; (2) An algorithm to generate test cases from these CFGs; (3) The application of a genetic algorithm to optimize and evaluate the coverage of the generated test cases; (4) An evaluation of the approach based on the criteria of minimality and coverage of the generated test cases through several case studies of commercial systems. The initial experimental results show that the generated test suite is minimal and covers all possible scenarios in the use case specifications. Therefore, we believe that this method of automatically generating test cases from specifications can be fully integrated into the TDD process, reducing development time and cost while ensuring that the software products are reliable and compliant with the specifications.

The rest of the paper is organized as follows. Section 2 provides a brief overview of related work. Our approach to improving TDD with automated test case generation from use case specifications is presented in Sect. 3. In Sect. 4, we provide experimental results to illustrate the effectiveness of the approach. Finally, we conclude the paper and suggest directions for future work in Sect. 5.

2 Related work

Test case generation has long been considered an essential process in the field of software engineering to ensure software quality and reliability, encompassing both code-based and specification-based approaches, among others. Recent advancements in automated test case generation (Alrawashdeh et al. 2019; Khurana et al. 2016; Ribeiro et al. 2018; Teixeira and Silva 2018; Taromirad and Runeson 2023) have leveraged various techniques to enhance software development practices, especially in the TDD process, where early-stage automation from requirements is critical.

One common direction is generating test cases directly from software specifications, such as use case specifications or Unified Modeling Language (UML) diagrams. For

instance, Alrawashdeh et al. (2019) propose an automated test case generation approach from use case descriptions, focusing on reducing test complexity and enhancing coverage. This method involves converting use case details into a control flow diagram and employing genetic algorithms for optimizing test cases. In another work, Khurana et al. (2016) present a comprehensive approach for generating and optimizing test cases using UML diagrams. Their methodology involves converting use case, activity, and sequence diagrams into respective graphs, integrating them into a system graph, and then applying genetic algorithms to optimize the generated test cases.

Moreover, Ribeiro et al. (2018) explore the application of model-based techniques for specifying requirements of real-time systems. They emphasize the importance of integrating UML, Systems Modeling Language (OMG 2010), and Modeling and Analysis of Real-Time and Embedded Systems (OMG 2011) to capture complex real-time system specifications accurately. This approach ensures that the generated test cases are aligned with the real-time constraints and requirements of the system, thereby improving the reliability and robustness of the software. Furthermore, Teixeira and Silva in Teixeira and Silva (2018) introduce the EasyTest approach, which integrates modeling, coding, and testing stages to reduce costs and development efforts. This approach emphasizes early defect detection in the modeling stage to prevent defects from being embedded in the coding stage. The EasyTest tool provides interoperability with the JUnit framework, facilitating the automatic generation of test cases from UML activity diagrams and ensuring comprehensive coverage through transition coverage criteria.

In contrast, Alagarsamy et al. (2023) proposed A3Test, a novel framework that combines assertions with automated test generation, significantly improving fault detection and assertion quality while addressing the limited adaptability of earlier code-based techniques. However, A3Test still relies on source code to generate assertions, limiting its suitability for early-stage TDD where code has yet to be developed. Additionally, Taromirad and Runeson (2023) presented a comprehensive literature survey on the use of assertions in software testing over the past two decades. Their study categorizes 119 research papers based on problem domains, proposed solutions, and evaluation strategies. Most solutions were implemented as prototype tools and evaluated on a limited scale, highlighting the need for further research on scalability and industrial applicability.

Recent developments in Artificial Intelligence (AI)-driven software testing have introduced more intelligent approaches to test generation. One notable approach is presented in Takerngsaksiri et al. (2024). This research introduces PyTester, a deep reinforcement learning-based framework that generates syntactically correct, executable, and effective test cases directly from natural language

requirements. By using a reward function considering syntax correctness, test executability, and code coverage, PyTester outperforms existing large language models like GPT-4.0 (OpenAI 2024) and StarCoder (Li et al. 2023) on key metrics such as requirement alignment, code coverage, and mutation score. This work is significant as it addresses the limitations of prior methods that primarily rely on code inputs rather than requirements, thus aligning more closely with the principles of TDD.

Amalfitano et al. (2023) conducted a comprehensive tertiary study analyzing how AI has been systematically applied to support different aspects of software testing. Their findings highlight a strong emphasis on the application of evolutionary algorithms, machine learning models, and knowledge-based techniques—especially in areas such as test case generation, test case optimization, and fault prediction. Olsthoorn (2022) proposed a framework that utilizes multiple tribes of AI, including grammar-based fuzzing and machine learning, to address challenges in generating highly structured test data and preserving method sequence patterns. While AI-driven approaches offer improved adaptability and structured input generation, they primarily serve dynamic system-level testing and lack explicit integration with early-stage requirement models like use cases, which are crucial for TDD adoption.

in Guo et al. (2024) investigate the capabilities of ChatGPT in automating code review tasks, specifically focusing on code refinement based on given code reviews. Their study demonstrates that ChatGPT outperforms existing state-of-the-art methods like CodeReviewer (Li et al. 2022) in terms of Exact Match and Bilingual Evaluation Understudy (Papineni et al. 2002) scores on code refinement tasks. This highlights the potential of leveraging advanced large language models for improving software quality and development efficiency. Finally, a broader survey by Baqar and Khanda (2024) echoed these concerns, noting key limitations of current AI-powered approaches, including limited domain adaptability, insufficient traceability, and weak integration with structured methodologies such as TDD.

In parallel, optimization-based approaches have also gained traction. For instance, Nayak et al. (2023) introduced an improved Grey Wolf Optimization (GWO) technique for test case prioritization, aiming to enhance regression testing efficiency. Although their method improves prioritization performance, it focuses on post-test-case-generation stages rather than the early generation of test cases from requirements. Likewise, Kumari et al. (2025) proposed a GWO-based framework for test case optimization, but similarly did not address specification-level generation from formal use case models—critical for aligning with early-stage TDD practices.

These recent contributions reinforce the need for solutions that not only automate test case generation from

early-stage artifacts like use case specifications but also provide measurable coverage and traceability for integration into test-driven methodologies. Our proposed approach addresses these limitations by offering a formalized, fully automated tool that transforms structured use case templates into optimized test cases guided by control flow models.

3 Approach

In this section, we describe our approach of combining automated test case generation from use case specifications with the TDD process. The approach consists of two tightly coupled components: (1) a modified TDD workflow enhanced with automated test generation, and (2) a technical framework for deriving test cases from use case specifications. The workflow, illustrated in Fig. 1, consists of two main components. The left side depicts the technical framework, starting with structured use case specifications in Cockburn format. These specifications are first transformed into CFGs to model the logical flow of actions and alternatives. A depth-first search (DFS) algorithm then extracts all possible execution paths from the CFGs. These paths are subsequently optimized using a genetic algorithm (GA) to reduce redundancy and enhance test coverage, ultimately generating a refined set of test cases. On the right side, the generated test cases drive the TDD cycle. This iterative process consists of: (1) adding a new test case, (2) implementing minimal code to pass the test, (3) verifying test results, and (4) refactoring as needed. The cycle repeats until all test

requirements are satisfied, ensuring systematic validation of the system. The two components are further elaborated in the following subsections. Section 3.1 explains how automated test generation is embedded within the TDD cycle, while Sect. 3.2 provides a detailed breakdown of the automated test case generation process, including CFG construction, test path generation, and optimization.

3.1 Applying automated test case generation in the TDD process

We propose a structured approach to enhance the TDD process through automatic test case generation, as illustrated in Fig. 2. It is designed to ensure that test cases are created early, based on well-defined use case specifications, and used as the primary driver for development activities. The process is organized into three main steps as follows:

Step 1: Requirement elicitation and use case specification

The process begins with gathering detailed system requirements from stakeholders. These requirements must be clear, measurable, and testable. Based on these, all possible use cases are identified and documented using the Cockburn template. Use cases are then prioritized according to business value and complexity. From this set, a high-priority use case is selected for implementation.

Step 2: Automated test generation

The selected use case is passed to the TCG, which automatically generates initial test cases. These test cases are refined to ensure they comprehensively cover different

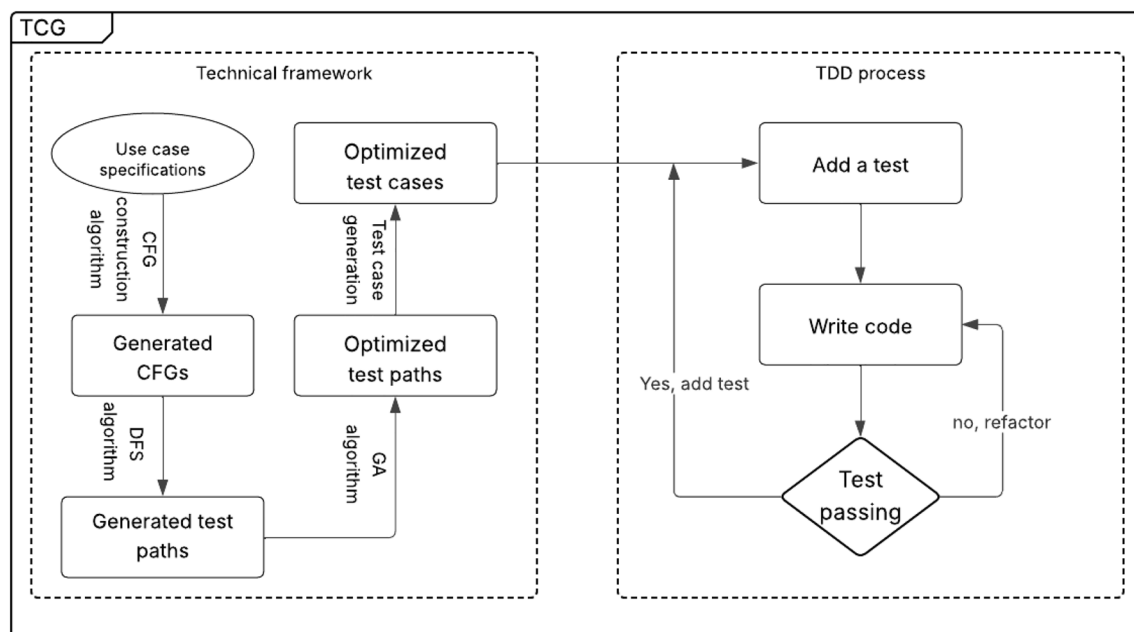


Fig. 1 Overview of the TCG framework integrated into the TDD process. The technical framework shows how test cases are automatically derived from use case specifications through CFG construction, DFS

traversal, and GA optimization. These test cases feed into the iterative TDD process, enabling early and automated testing from requirements

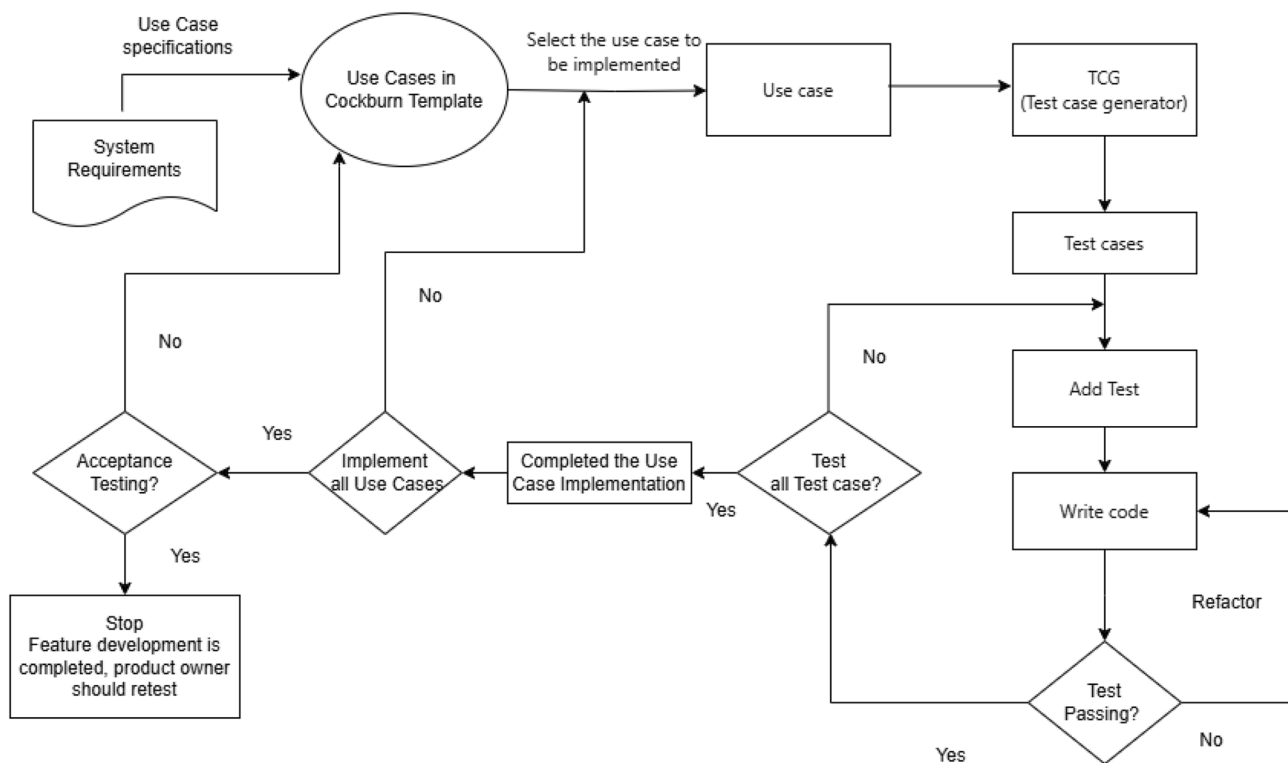


Fig. 2 Applying automated test case generation in the TDD process

execution scenarios and conditions within the use case. The output from this step provides the foundation for guiding development in the next phase.

Step 3: TDD execution and iteration

Development begins by writing code to satisfy the generated test cases. If any test fails, the code is refactored and re-tested until all tests pass. Additional unit tests can be added during this phase. Once implementation is completed for a use case, the process repeats with the next prioritized use case. After all use cases are implemented, acceptance testing is conducted with stakeholders. If the system meets all requirements, it is approved for release. The product is then deployed to the production environment, where it is monitored and maintained regularly. This structured approach ensures that each feature is thoroughly tested and validated, resulting in a robust and reliable system that meets stakeholder expectations and user needs.

3.2 Automated test case generation

Automated test case generation is at the core of our approach. It includes the following modules: CFG construction, test path generation, and optimization. Figure 3 illustrates the overall process for automated test case generation. The process begins with structured use case specifications, depicted at the bottom left of the figure. From these specifications, two distinct paths are possible:

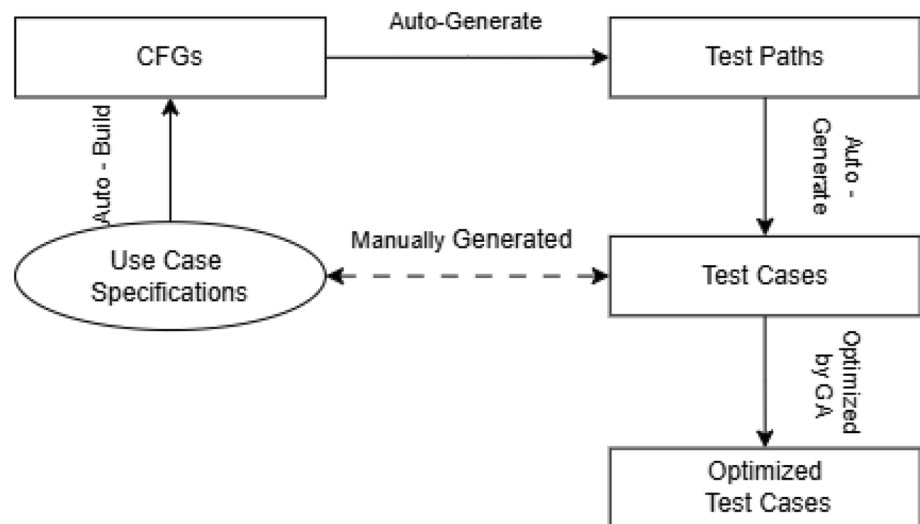
- **Automated path (solid lines):** The system automatically constructs CFGs from the use case specifications. Test paths are then generated from these CFGs and subsequently transformed into test cases. These test cases undergo further refinement using a genetic algorithm, resulting in optimized test cases.
- **Manual path (dashed line):** Alternatively, testers can manually derive test cases directly from the use case specifications. These manually created test cases are then processed by the same optimization module to maintain consistency and ensure quality.

This hybrid approach combines the efficiency of automation with the flexibility of manual testing. However, as indicated by the dashed lines in the figure, the manual process requires more effort and may lack the precision offered by the automated flow. To better understand its structure and rationale, the approach can be summarized as follows:

Step 1: CFG construction

The CFG construction module begins with collecting use case specifications, which serve as the foundation for building CFGs. These graphs represent the logical flow of the system and detail all possible paths through the use cases. This step ensures a thorough understanding and documentation of the system requirements.

Fig. 3 Automated test case generation approach, showing both automatic and manual test creation paths, and the optimization stage using a genetic algorithm



Step 2: Test path generation

In the test path generation module, test paths are generated automatically. These paths cover all potential scenarios and conditions defined in the use cases, ensuring comprehensive coverage. These generated test paths are then used to create initial test cases.

Step 3: Optimization

An optimization module is used to optimize the test cases. This optimization process refines the test cases by eliminating redundancies and improving their ability to detect

potential issues in the code. The result is a set of optimized test cases that provide better test coverage with fewer tests.

3.2.1 Constructing CFGs from use case specifications

In this module, we propose an algorithm, detailed in Algorithm 1, and implement it in Python to automatically generate CFGs from use case specifications.

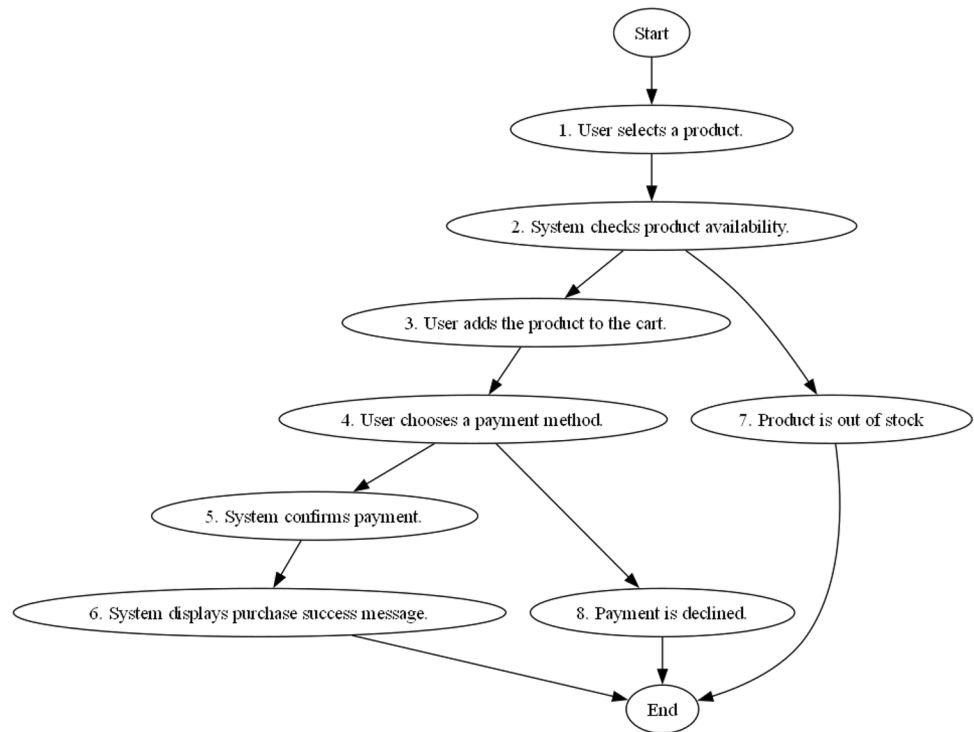
Input: Use case specification in Cockburn format

Output: Control Flow Graph (CFG)

```

1: Initialize CFG with root node  $R$ 
2:  $current \leftarrow R$ ,  $node\_counter \leftarrow 1$ 
3: for each  $step$  in main success scenario do
4:   Create node  $n$  labeled with  $step$ 
5:   Add  $n$  to CFG
6:   if  $node\_counter = 1$  then
7:     Add edge  $R \rightarrow n$ 
8:   else
9:     Add edge  $previous \rightarrow n$ 
10:  end if
11:   $previous \leftarrow n$ 
12:   $node\_counter \leftarrow node\_counter + 1$ 
13: end for
14: Create node  $E$  labeled "End" and add to CFG
15: Add edge  $previous \rightarrow E$ 
16: for each  $(p, ext)$  in extension steps do
17:   Create node  $n_{ext}$  labeled with  $ext$ 
18:   Add  $n_{ext}$  to CFG
19:   Add edge from  $main\_steps[p] \rightarrow n_{ext}$ 
20:   Add edge  $n_{ext} \rightarrow E$ 
21:    $node\_counter \leftarrow node\_counter + 1$ 
22: end for
23: return CFG
  
```

Algorithm 1 Constructing CFGs from use case specifications

Fig. 4 Generated CFG from online purchase use case specification**Table 1** Online purchase use case specification

Name	Online purchase
Goal	Allow customers to purchase products on an e-commerce website
Actors	Customer, Website System
Preconditions	Customer has an active account and is logged in
Postconditions	Order is completed, and confirmation is displayed
Invariants	Product availability must be ensured before proceeding with the purchase
Main success scenario	1. User selects a product 2. System checks product availability 3. User adds the product to the cart 4. User chooses a payment method 5. System confirms payment 6. System displays purchase success message
Variations	None
Extensions	2. Product is out of stock 4. Payment is declined
Included use cases	Login, View product details, Manage shopping cart, Payment

The algorithm's correctness is demonstrated through its ability to systematically generate a CFG from the main success scenario and associated extension points in the use case. It starts by initializing a root node labeled "R" and sequentially processes each step from the main success

scenario. For each step, the algorithm creates a node and appends it to the graph, ensuring that it connects to the previous step, forming a coherent and sequential control flow. When an extension point is encountered, the algorithm creates a branching node from the corresponding step in the main success scenario. This branch represents alternative paths and leads to an "E" end node, maintaining the structural integrity of the CFG. The algorithm ensures that the textual labels for each node accurately represent the specific step or extension in the use case, preserving the semantic meaning of the original use case specification. As each step is processed, the algorithm updates the current node to maintain control flow consistency. After processing all steps, the algorithm finalizes the CFG by connecting the last node to the end node "E".

In addition, the algorithm processes each statement in the use case specification exactly once, making its time complexity $\mathcal{O}(n)$, where n is the total number of steps, including both the main success scenario and the extension points. Each step involves creating nodes and edges, which are constant-time operations, ensuring that the overall time complexity remains linear with the number of steps. This makes the algorithm highly efficient and scalable for large use case specifications.

The CFG shown in Fig. 4 was generated from the Online Purchase use case specification, as described in Table 1,

using the provided algorithm. Initially, the CFG is created with a root node *start*. The steps from the use case are extracted and converted into corresponding nodes in the CFG. These steps include selecting a product, adding it to the shopping cart, proceeding to checkout, entering shipping details, and system processes payment. Each step is represented as a node in the CFG, with directed edges linking them to show the sequence of actions. Additionally, conditions such as the product being out of stock and invalid payment information are handled by creating conditional nodes and corresponding branches. Finally, all nodes are connected to form a complete CFG, representing the entire online purchase process.

In comparison to other algorithms for generating CFGs, such as those based on code or bytecode analysis (Das et al. 2024; Le 2020), which focus on representing control flows at a code level, typically working with loops, conditionals, and function calls within a program, these approaches often emphasize low-level code constructs and require more sophisticated parsing techniques. Whereas, our algorithm emphasizes high-level actions and decision points described in the use case, making it more suitable for business logic and system behavior analysis.

One notable difference is that our algorithm directly maps the sequence of actions in a use case into the CFG, including

extensions and conditional branches, which align more naturally with business scenarios. On the other hand, Abstract Syntax Tree (AST)-based CFG generation methods (Koppel et al. 2022; Putra et al. 2021) require parsing code, which adds complexity when inferring high-level business logic. Moreover, CFG generation from bytecode (Alekseev et al. 2012), such as those used in Java or Python, can capture intricate details of program execution, but they often overlook the semantic meaning behind higher-level processes, such as user interaction flows or system responses. Our algorithm, being tailored to use cases, retains the semantic integrity of the process steps, making the graph more interpretable and actionable for stakeholders who are not necessarily involved in the technical implementation.

3.2.2 Test paths generation from CFGs

In this module, we provide an algorithm, as shown in Algorithm 2, which uses DFS to generate test paths from CFGs. The algorithm starts from the 'Root' node of the CFG and employs a recursive DFS approach to explore all possible paths until it reaches the 'End' node. The resulting set of test paths helps identify test cases for each execution path in the scenario.

```

1: function GENERATE_TEST_PATHS(control_flow_graph)
2:   test_paths ← empty list
3:   if "Root" ∈ control_flow_graph then
4:     DFS("Root", [], set())
5:   end if
6:   return test_paths
7: end function
8: function DFS(node, path, visited)
9:   visited ← visited ∪ {node}
10:  path ← path ∪ {node}
11:  if node = "End" then
12:    test_paths ← test_paths ∪ {copy(path)}
13:  else
14:    for all next_node ∈ control_flow_graph[node]["next"] do
15:      if next_node ∉ visited then
16:        DFS(next_node, copy(path), copy(visited))
17:      end if
18:    end for
19:  end if
20: end function

```

Algorithm 2 Generating test paths using DFS

Table 2 Test paths generated from the CFG in Fig. 4

No.	Test Paths	Test Cases
1	Start-1-2-3-4-5-6-End	Pre.: Logged in. 1: (A: Select product, EO: Product is selected, availability checked)→ 2: (A: Check availability, EO: Product available)→ 3: (A: Add to cart, EO: Product added) → 4: (A: Choose payment, EO: method selected)→ 5: (Confirm payment, EO: Payment success)→ 6: (A: Display Message, EO: Success shown). Post.: Order complete, system stable
2	Start-1-2-7-End	Pre.: Logged in. 1: (A: Select product, EO: confirmed)→ 2: (A: Check stock, EO: out of stock)→ 7: (A: Notify, EO: Unavailable notified). Post.: No order placed
3	Start-1-2-4-8-End	Pre.: Logged in. 1: (A: Select product, EO: added to cart)→ 2: (A: Check stock, EO: Available)→ 4: (A: Add to cart, EO: Updated)→ 8: (A: Attempt payment, EO: declined). Post.: Declined notified

Note: A: Action; EO: Expected Output; Pre.: Preconditions; Post.: Postconditions

In terms of evaluation, the algorithm's effectiveness is highlighted by its ability to handle the typical size of CFGs encountered in practice. Given that the number of events in a use case is typically small, and in practical large-scale scenarios, use cases are often decomposed into smaller, more manageable components, the number of nodes in CFGs usually remains relatively small. As a result, the DFS-based algorithm proves to be efficient for such cases. By recursively visiting each node and marking it as visited,

the algorithm ensures that all possible paths are explored comprehensively. Overall, this thorough approach guarantees that the generated test paths provide maximum coverage, which is crucial for identifying potential issues in the software.

Table 2 depicts the test paths generated from the CFG in Fig. 4 using Algorithm 2. The algorithm employs DFS to explore all possible paths from the root node to the end node within the control flow graph. These paths ensure comprehensive testing by covering all nodes and edges, including all conditional branches. This results in multiple test paths that encapsulate various execution sequences through the CFG, providing thorough coverage of the system's behavior under different conditions. By following these paths, testers can verify that all potential scenarios and transitions within the system are correctly handled, ensuring robust and reliable software performance.

3.2.3 Optimizing generated test paths

In this module, we propose a genetic algorithm to optimize test paths, ensuring that the number of test paths is minimized while achieving maximum coverage (Algorithm 3). The input to the algorithm is a set of test paths generated from the CFGs, and the output is an optimized set of test paths. The algorithm iteratively evolves the population of test paths through selection, crossover, and mutation operations, guided by the fitness scores obtained from evaluating each test path. This technique aims to reduce redundancy in test execution and improve testing efficiency without compromising fault detection capability.

```

1: function GENETIC_ALGORITHM(test_paths, population_size, max_generations, mutation_rate)
2:   population ← INITIALIZE_POPULATION(test_paths, population_size)
3:   for generation ← 1 to max_generations do
4:     fitness_scores ← EVALUATE_POPULATION(population, test_paths)
5:     new_population ← SELECT_FITTEST_INDIVIDUALS(population, fitness_scores)
6:     while length(new_population) < population_size do
7:       parent1, parent2 ← SELECT_PARENTS(new_population, fitness_scores)
8:       offspring ← CROSSOVER(parent1, parent2)
9:       if random() < mutation_rate then
10:        offspring ← MUTATE(offspring, test_cases)
11:       end if
12:       new_population ← new_population ∪ {offspring}
13:     end while
14:     population ← new_population
15:   end for
16:   return SELECT_BEST_INDIVIDUAL(population, test_paths)
17: end function

```

Algorithm 3 Genetic algorithm for test path optimization

Table 3 Illustration of test path evolution with valid CFG paths

Generation	Test path	Operation	Fitness
Gen 0	P ₁ : A→B→C	Initial Population	60%
	P ₂ : A→D→E	Initial Population	60%
Gen 1	P ₃ : A→B→D→E	Crossover (P ₁ × P ₂ , repaired)	70%
Gen 5	P*: A→B→D→C→End	Best Individual (Optimized)	90%

The algorithm involves three main steps as follows:

Step 1: Initialization of the population

The algorithm first checks if the list of test paths is empty. If so, it returns an empty list. Otherwise, it initializes a population of individuals using random integers, structuring them as sequences representing test paths. Each individual corresponds to a candidate solution for test path optimization. These individuals are evaluated using a fitness function, which calculates the proportion of control flow graph (CFG) transitions (edges) covered by the test path. The fitness is computed as:

$$\text{Fitness} = \left(\frac{\text{Number of covered edges}}{\text{Total number of edges in the CFG}} \right) \times 100 \quad (1)$$

This measure favors individuals that provide broader coverage and guides the evolutionary process toward more effective test paths.

Step 2: Evolutionary process

The algorithm runs through a number of generations. In each generation, the algorithm performs the following:

1. The population is evaluated using a fitness function, which assesses each individual's effectiveness based on

its transition coverage. The fitness score is stored for each individual to guide selection.

2. The algorithm selects the fittest individuals—those with the highest fitness scores—for retention and reproduction. Selection techniques such as tournament or roulette wheel are used to ensure that better-performing individuals are more likely to be chosen.
3. Two individuals are selected as parents based on their fitness scores. These parents participate in the crossover operation.
4. Crossover is performed between the parent individuals to generate a new offspring. For instance, a child path may be created by combining the first part of one parent (e.g., A → B) with the latter part of the other (e.g., D → E), resulting in a new path like A → B → E. However, if this path is not valid in the CFG (e.g., B is not connected to E), the algorithm applies repair mechanisms or regenerates a valid path (e.g., A → B → D → E).
5. Mutation is applied to the offspring to introduce variation and prevent premature convergence. Mutation may involve altering a node (e.g., C → D), inserting a new step, or removing a redundant one, ensuring that the path remains feasible in the CFG. If a mutation leads to an invalid path, the algorithm either discards the individual or applies repair techniques to restore CFG validity. Invalid paths are discarded or repaired.

The values of the algorithm's hyperparameters are set as follows: population_size = 20 (to balance diversity and computational efficiency), mutation_rate = 0.1 (to introduce occasional variation without destabilizing good solutions). Although intermediate generations are not always shown, the evolutionary process continues through several iterations. For example, at Generation 0, two initial individuals P₁ and P₂ (A→B→C and A→D→E) might each have a

Fig. 5 Architecture of the TCG tool

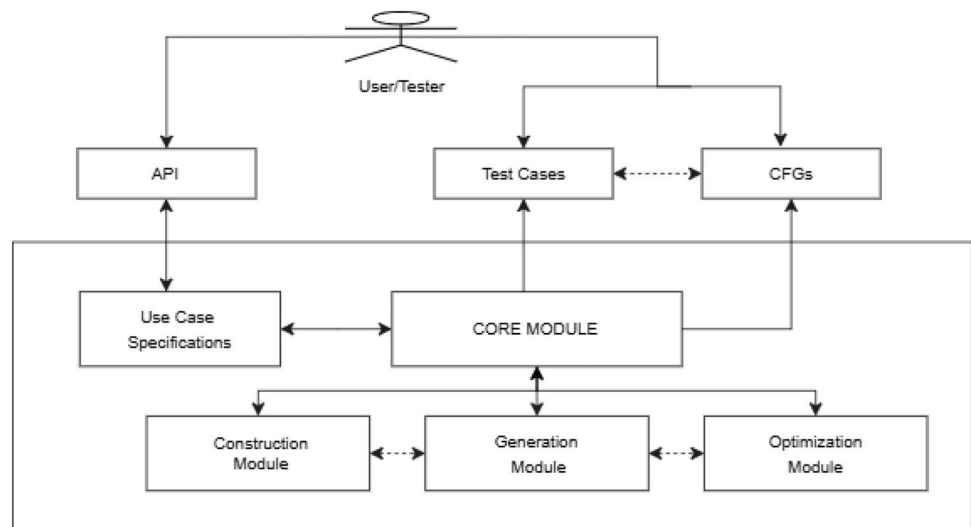


Table 4 Optimal test-path after 100 generations of the CFG in Fig. 4

Parameter/Test-path	Value/Test-path	Description
Generation	100	Number of generations
Fittest	3	Number of the fittest individual
Test-Path 1	Start → 1 → 2 → 3 → 4 → 5 → 6 → End	Path covering maximum edges

fitness of 60%. After crossover and mutation, new individuals such as P_3 ($A \rightarrow B \rightarrow E$) and P_4 ($A \rightarrow B \rightarrow D$) emerge with higher fitness ($\sim 70\%$). At Generation 5, the best individual P^* ($A \rightarrow B \rightarrow D \rightarrow \text{End}$) achieves 90% coverage.

Step 3: Returning the result

After completing the specified number of generations, the algorithm selects the best individual based on the highest fitness score. This individual is returned as the optimized test path. While P^* may not achieve 100% coverage, it significantly improves coverage and efficiency over earlier generations. The remaining uncovered transitions often correspond to rare or low-probability paths such as error handling. These may be reincorporated if complete path coverage is required. Table 3 illustrates the improvement of test path quality over generations, showing how crossover and mutation contribute to the discovery of high-coverage test paths.

Regarding convergence and time complexity, the genetic algorithm iteratively improves the population of test cases over a specified number of generations. Each generation involves evaluating the fitness of individuals, selecting parents, and applying crossover and mutation operations. The time complexity of the algorithm is influenced by the population size, the number of generations, and the complexity of the fitness evaluation function. In the TDD software development process, use cases are often specified in a manner that makes them small enough to be implemented easily. This practical consideration allows developers to maintain clarity and manageability in their testing efforts. Given this context, choosing a population size of 20 in our genetic algorithm is effective. The population size aligns well with the expected number of test cases, ensuring sufficient diversity without overwhelming complexity.

For instance, Table 2 represents three generated test paths that cover all possible execution sequences through the CFG of Fig. 4. This result shows that test path number one is the best path to cover maximum transactions in the control flow graph of the Online Purchase use case.

Table 5 Statistics on generated test paths from use case specifications of an E-Commerce system

No.	Use case	Generated test paths	Index of optimized test paths	Percentage of transition coverage
1	Order tracking	5	2	75%
2	Process payment	4	1	65%
3	Product search	7	5	55%
4	Purchase	3	3	80%
5	Registration	4	1	75%
6	Review submission	3	3	80%

4 Experimental results

In this section, we present the architecture of the TCG tool, followed by experimental results and a discussion on both the effectiveness and limitations of the TCG tool.

4.1 A support tool

We developed a TCG tool to validate the proposed approach. The architecture of this tool, as shown in Fig. 5, includes detailed implementation aspects. These aspects cover all the algorithms presented in Sect. 3 for converting use case specifications to CFGs, as well as the methods for generating and optimizing test cases. The full source code of the TCG tool is available on GitHub.¹

The TCG tool not only enables the automatic generation of test cases but also supports their optimization through advanced algorithms. By reducing redundancy and ensuring comprehensive coverage, the tool enhances the efficiency of the testing process. Furthermore, the TCG tool offers an intuitive, user-friendly interface that simplifies interaction. Users can easily specify or upload use case documents, visualize the corresponding CFGs, test paths, and view the generated test cases. This interface makes the tool highly accessible for software developers and testers, allowing them to integrate automated test case generation into their TDD workflows.

4.2 Case study and results

We use the use case specifications in the e-commerce system to test the toolkit. The test results are evaluated through the stages of generating CFG using Algorithm 1, followed by the identification of all possible test paths via Depth-First Search (Algorithm 2). This approach exhaustively identifies every potential execution sequence from the root to the end node in the CFG, covering all conditional branches and

¹ <https://github.com/binhtt/TCG-TOOL/tree/main>.

Table 6 Comparison of traditional and TCG approaches

Projects	Use cases	Traditional approach				TCG approach			
		Test cases	Gen. Time (h)	Trans. Cov. (%)	Errors Det.	Test cases	Gen. Time (h)	Trans. Cov. (%)	Errors Det.
E-comm. Sys	26	144	52.4	65	23	135	0.43	83	27
Online Bank	21	148	54	67.5	33	143	0.35	73	43
Ride-Sharing	20	141	51.3	64	19	131	0.33	64	19
Supply Chain	25	156	56.7	70	15	147	0.42	87.5	17
Mobile Pay	20	141	51.3	69	21	131	0.33	51.3	21

Note: Gen. Time (h): Test generation time in hours; Trans. Cov. (%): Transition coverage; Errors Det.: Errors detected

Table 7 Statistical comparison between traditional and TCG approaches across five projects

Metric	Traditional mean	TCG Mean	<i>p</i> value	Cohen's <i>d</i>
Test generation time (h)	53.14	0.37	< 0.001	3.5 (very large)
Transition coverage (%)	67.1	71.76	0.008	1.04 (large)
Errors detected	22.2	25.4	0.004	1.18 (large)

Note: Statistical tests were performed using paired *t*-tests. Effect size interpretations follow (Cohen 1988) and (Sawilowsky 2009)

process steps to establish test cases. Subsequently, Algorithm 3, a genetic algorithm, was employed to optimize these test paths. The genetic algorithm refines the initial set of paths through iterative processes of selection, crossover, and mutation. This optimization aims to enhance the efficiency of testing by maximizing coverage and minimizing execution time. The fitness of each test case was evaluated based on these criteria, enabling the selection of the most effective test paths.

Table 4 illustrates the results of optimizing the test paths of the CFG in Fig. 4. After 100 generations, there are three test paths, with test path 1 being the best test path to cover the maximum transactions of the CFG. This optimized path retains comprehensive coverage of the CFG while reducing redundancy, thereby streamlining the testing process. The results indicate that the genetic algorithm effectively reduces the number of test cases needed without sacrificing thoroughness, making the approach both efficient and robust for practical applications in software testing.

Table 5 presents the experimental results of generating and optimizing test paths from the use case specifications of an e-commerce system. For instance, three test paths were generated from the Purchase use case, with the optimized path (index three) achieving 80% transition coverage in the corresponding control flow graph (CFG). This percentage signifies that the optimized path traverses 80% of all control flow transitions (edges) in the CFG derived from the use case. The remaining 20% primarily involve rarely executed branches, such as exception handling or low-probability alternatives, which were omitted during optimization to maintain a compact test suite. Although these transitions are

not inherently insignificant, the genetic algorithm prioritizes frequently executed and impactful paths to maximize practical efficiency. In critical systems requiring exhaustive testing, these paths can be reincorporated to achieve full-path or branch-level coverage. Transition coverage is widely recognized as an effective metric for evaluating test adequacy, particularly when strong traceability from requirements to implementation is essential.

To evaluate the effectiveness of this toolkit, we developed five software application systems at the Software Engineering Lab of Phenikaa University, following the TDD model. The experiment involved creating test cases using the traditional approach with three testers, each with over five years of experience, and the TCG toolkit with one tester. The experimental results are presented in the Table 6. This table provides a comparative analysis of the traditional and TCG approaches across several projects. Key benefits of using the TCG approach include a smaller number and shorter time required to build test cases, while still ensuring better coverage and detecting more errors compared to the traditional approach. For instance, with the manual approach, the E-commerce System (E-comm. Sys) project, which has 26 use cases, generated 144 test paths, executed by three testers in 52.4 h, achieving 65% transition coverage and detecting 23 errors. In contrast, with the TCG approach, 135 test paths were generated, and executed by one tester in 0.43 h, achieving 83% transition coverage and detecting 27 errors.

To evaluate the effectiveness of the proposed TCG approach in comparison to traditional methods, we conducted statistical significance tests across three key metrics: test generation time, transition coverage, and the number of detected errors. These metrics were collected from five representative software projects detailed in Table 6. To assess whether the performance differences between the traditional and TCG approaches were statistically significant, we applied paired *t*-tests. The results in Table 7 show *p*-values below 0.01 for all metrics, indicating that the observed improvements with TCG are highly unlikely to have occurred by chance. Specifically, TCG achieved a substantial reduction in test generation time (mean: 0.37 h vs. 53.14 h), higher transition coverage (mean: 71.76% vs.

67.1%), and a greater number of detected errors (mean: 25.4 vs. 22.2).

In addition, we computed effect sizes using Cohen's d , a standardized measure of practical significance. The results revealed very large effects for test generation time ($d = 3.5$) and large effects for both transition coverage ($d = 1.04$) and error detection ($d = 1.18$). According to established thresholds, an effect size of $d \geq 0.8$ is considered large, indicating that TCG not only yields statistically significant results but also offers meaningful practical advantages. These findings empirically validate the superiority of TCG in terms of testing efficiency, coverage, and fault detection, supporting its applicability in modern software development workflows.

4.3 Discussion

The TCG tool was evaluated based on its effectiveness, efficiency, and integration capabilities in generating and optimizing test cases from use case specifications. The assessment focused on key criteria: test case optimization, user experience, and alignment with the TDD process. Compared to recent tools such as PyTester and EasyTest, the proposed TCG approach offers a different set of trade-offs. PyTester focuses on generating test cases from natural language requirements using deep reinforcement learning, achieving high mutation scores and syntax correctness. However, it requires substantial training data, fine-tuning, and lacks seamless integration with structured TDD workflows. EasyTest, on the other hand, emphasizes transition coverage from UML activity diagrams, but does not perform test optimization or support traceability from Cockburn-style use cases.

In addition to PyTester and EasyTest, notable code-based tools such as EvoSuite (Fraser and Arcuri 2011) and Randoop (Pacheco and Ernst 2007) have demonstrated strong capabilities in automatically generating unit tests from Java bytecode. EvoSuite uses genetic algorithms to evolve test suites that maximize coverage, while Randoop generates sequences of method calls using feedback-directed random testing. However, both tools operate post-implementation, requiring source code to be available. This limits their compatibility with early-stage TDD, where code does not yet exist.

In contrast, TCG provides a lightweight, specification-driven solution with explicit alignment to TDD. While it achieves high transition coverage (up to 87.5%) and detects more faults in our experiments, its advantage lies in the compactness and traceability of test cases from early-stage artifacts. A direct comparison across fault detection and test effort is shown in Table 6. Although these results favor TCG, further benchmark studies with diverse systems are needed to fully contextualize its performance.

Test case optimization: The TCG tool's optimization module, which uses a genetic algorithm, significantly reduced the number of test cases while maintaining high coverage. This optimization resulted in a more efficient set of test cases, saving time and resources during testing. For example, it optimized three test cases into one for the Purchase use case, demonstrating the tool's ability to optimize the testing process while achieving 80% transition coverage. This high coverage is essential for reducing undetected errors and ensuring robust software testing.

While transition coverage serves as a key metric in this study due to its direct relevance to control flow completeness and alignment with use case scenarios, we acknowledge that relying solely on this criterion may not capture all aspects of test effectiveness. Future evaluations can incorporate additional coverage metrics such as branch coverage, path coverage, and mutation score to assess the fault-detection ability and thoroughness of the generated test suites. These complementary criteria would provide a more holistic assessment of the test quality and help compare with alternative test generation methods.

User-friendly interface and ease of integration: The TCG tool features an intuitive interface that allows users to easily upload use case documents, view generated test cases, and visualize CFG and test paths. This design ensures quick adoption and integration into existing TDD workflows, supporting continuous testing and refinement. Although currently focused on functional aspects, the interface and workflow of TCG could be extended to support non-functional testing in the future, such as integrating modules for performance profiling or static security analysis, which would make it more applicable to real-world development environments.

Alignment with TDD principles: By automating the generation of test cases from use case specifications, the TCG tool aligns well with the TDD process. This automation reduces manual effort, allowing developers to focus on writing and refining code. The generated test cases are closely aligned with initial requirements, maintaining the integrity of the TDD process. At present, TCG aligns with functional test generation based on control flow and transition coverage. Non-functional testing aspects such as performance, scalability, or security are outside the current scope. However, the generated test cases can serve as a basis for stress testing (e.g., by scaling input volumes), and the CFG structure could be extended to model performance-critical paths or security-sensitive transitions. Future work includes integrating runtime metrics (e.g., response time, memory usage) into the optimization criteria and exploring how TCG could be combined with static security analyzers to identify vulnerability-prone paths early in development.

Table 8 Summary of TCG benefits and limitations

Aspect	Benefits	Limitations
Test automation	Reduces manual effort using use case-driven test generation	Dependent on well-structured input specifications
Optimization	Genetic algorithm minimizes test cases while maintaining high coverage	Scalability in large, complex, or real-time systems remains unverified
Coverage and fault detection	Achieves 80–87.5% transition coverage (Table 6); detects more errors on average (Table 7)	Optimization may omit rare edge cases; lacks support for non-functional testing
Integration with TDD	Seamless alignment with early-stage TDD; tests derived directly from requirements	Currently focused on functional aspects; lacks coverage for performance/security
Tool usability	User-friendly interface, intuitive workflow, and open-source implementation	Lacks empirical data on industrial adoption and long-term usage

To further evaluate the tool's practical applicability in real-world environments, future work will involve structured usability studies with professional developers. These studies will investigate how easily the TCG tool integrates into existing development workflows, including aspects such as learnability, developer satisfaction, and impact on overall testing efficiency. Observing developer interactions in real-world TDD projects will also inform improvements to the tool's interface and integration capabilities. These usability insights are essential for validating the tool not only from a technical but also from a human-centered perspective.

Scalability and limitations: Despite its strengths, the TCG tool has several limitations. It may encounter difficulties with larger and more complex use cases due to the significant computational resources required for exhaustive search and optimization. Moreover, the current evaluation does not include highly complex systems, such as distributed or real-time applications. As such, the effectiveness and scalability of the genetic algorithm in those environments remain unverified. In particular, when dealing with real-time or distributed systems, where interactions are concurrent and dynamic, the current static CFG-based model may not fully capture system behaviors. Addressing such environments may require incorporating runtime monitoring or distributed optimization strategies.

While TCG has demonstrated efficacy in small-to medium-scale systems, its behavior in large, enterprise-level applications warrants further exploration. For example, in a distributed microservices architecture with hundreds of interdependent use cases, the computational complexity of

CFG construction and genetic algorithm optimization could increase exponentially. To mitigate this, future iterations might employ distributed computing frameworks (e.g., Apache Spark (Zaharia et al. 2010)) to parallelize CFG traversal or adopt incremental optimization strategies that prioritize critical paths. Additionally, integrating TCG with runtime monitoring tools could enable dynamic adaptation of test cases to evolving workflows, such as handling real-time inventory updates in global e-commerce platforms. While these are hypothetical extensions, they illustrate TCG's potential adaptability to enterprise contexts, provided computational and architectural challenges are addressed.

To summarize the empirical findings and practical trade-offs of TCG, Table 8 presents an overview of its main benefits and limitations across key dimensions.

5 Conclusion and future work

In this paper, we present a comprehensive approach to improving TDD through automated test case generation from use case specifications. The approach transforms use case specifications into CFGs, generates test paths via DFS, and optimizes them using a genetic algorithm. This automation reduces manual effort, aligns tests more closely with requirements, and generates optimized test cases covering essential scenarios, thereby enhancing testing efficiency and manageability.

The experimental results indicate that TCG achieves transition coverage ranging from 80% to 87.5% in the Purchase and Supply Chain use cases, while reducing test generation time by approximately 99% in the E-commerce system, as shown in Tables 5 and 6. Nevertheless, several challenges remain. In particular, the scalability of the approach has yet to be verified for large-scale or distributed systems, where exhaustive CFG traversal and genetic optimization could introduce significant computational overhead. Furthermore, reliance on static analysis constrains adaptability to dynamic behaviors and evolving requirements.

Future research should focus on enhancing the scalability, adaptability, and applicability of the proposed approach across diverse and complex software environments. Specifically, evaluating the convergence behavior and computational efficiency of the genetic algorithm in large-scale or distributed systems remains an essential next step. Additionally, integrating dynamic analysis techniques could complement the current static analysis model, enabling more comprehensive validation of test cases in real-time or evolving execution contexts.

Building on this foundation, we aim to enhance the TCG tool through several advanced directions: (a) applying natural language processing (NLP) techniques to automatically interpret free-text use case descriptions; (b) developing hybrid optimization strategies that combine genetic algorithms with reinforcement learning to improve the adaptability of test path generation; and (c) scaling the empirical evaluation to industrial-scale software systems to validate robustness and effectiveness under realistic development constraints.

Declarations

Conflict of interest The authors have no Conflict of interest to declare that are relevant to the content of this article.

Financial or proprietary interests The authors have no financial or proprietary interests in any material discussed in this article.

References

- Alagarsamy S, Tantithamthavorn C, Aleti A (2023) A3Test: assertion-augmented automated test case generation. *Inf Softw Technol*. <https://doi.org/10.2139/ssrn.4724885>
- Alekseev S, Dhanraj V, Reschke S, Palaga P (2012) Tools for control flow analysis of java code. *Parallel and distributed computing and systems*
- Alrawashdeh T, Almomani DA, Thunibat A, Tamimi A (2019) An automated approach to generate test cases from use case description model. *Comput Model Eng Sci* 119:409–425
- Amalfitano D, Faralli S, Hauck JCR, Matalonga S, Distanto D (2023) Artificial intelligence applied to software testing: a tertiary study. *ACM Comput Surv* 56(3):1–38
- Astels D (2003) *Test driven development: a practical guide*. Prentice Hall Professional Technical Reference, Upper Saddle River, NJ
- Baqar M, Khanda R (2024) The future of software testing: AI-powered test case generation and validation. *arXiv:https://arxiv.org/abs/2409.05808*
- Beck (2002) *Test driven development: by example*. Addison-Wesley Longman Publishing Co., Inc., USA
- Beck K, Beedle M, Bennekum A, Cockburn, A, Cunningham W, Fowler M, Grenning J, Highsmith J, Hunt A, Jeffries R, Kern J, Marick B, Martin RC, Mellor S, Schwaber K, Sutherland J, Thomas D (2001) *Manifesto for Agile Software Development*
- Cohen J (1988) *Statistical power analysis for the behavioral sciences*, 2nd edn. Lawrence Erlbaum Associates, Hillsdale, NJ
- Das D, Mathews NS, Mathai A, Tamilselvam S, Sedamaki K, Chimalakonda S, Kumar A (2024) COMEX: a tool for generating customized source code representations. *ASE '23*. IEEE Press, Echternach, Luxembourg
- Erdogmus H, Morisio M, Torchiano M (2005) On the effectiveness of the test-first approach to programming. *IEEE Trans Softw Eng* 31(3):226–237
- Fraser G, Arcuri A (2011) Evosuite: automatic test suite generation for object-oriented software. In: *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on foundations of software engineering. ESEC/FSE '11*. Association for Computing Machinery, New York, NY, USA, pp 416–419
- Guo Q, Cao J, Xie X, Liu S, Li X, Chen B, Peng X (2024) Exploring the potential of ChatGPT in automated code refinement: an empirical study. *ICSE '24*. Association for Computing Machinery, New York
- Janzen D, Saiedian H (2005) Test-driven development concepts, taxonomy, and future direction. *Computer* 38(9):43–50
- Khurana N, Chhillar RS, Chhillar U (2016) A novel technique for generation and optimization of test cases using use case, sequence, activity diagram and genetic algorithm. *J Softw* 11:242–250
- Koppel J, Kearn J, Solar-Lezama A (2022) Automatically deriving control-flow graph generators from operational semantics. *Proc ACM Program Lang* 6(ICFP):742–771
- Kumari S, Jindal S, Sharma A (2025) Test case optimization using grey wolf algorithm. *Softw Qual J* 33(1):20
- Le K (2020) Automating control-flow graph generation from source code. Technical report, School of Engineering and Applied Science, University of Virginia
- Li R, Allal LB, Zi Y, Niklas Muennighoff et al (2023) StarCoder: may the source be with you!
- Li Z, Lu S, Guo D, Duan N, Jannu S, Jenks G, Majumder D, Green J, Svyatkovskiy A, Fu S, Sundaresan N (2022) Automating code review activities by large-scale pre-training
- Nayak G, Barisal SK, Ray M (2023) CGWO: an improved grey wolf optimization technique for test case prioritization. *Prog Comput Softw* 49(8):942–953
- Olsthoorn M (2022) More effective test case generation with multiple tribes of AI. In: *2022 IEEE/ACM 44th international conference on software engineering: companion proceedings (ICSE-Companion)*, pp 286–290
- OMG (2010) *Systems Modeling Language (SysML) Specification – Version 1.1*. Technical report, Object Management Group (OMG)
- OMG (2011) *UML Profile for MARTE: Modeling and Analysis of Real-Time Embedded Systems, Version 1.1*. Technical report, Object Management Group (OMG)
- OpenAI: GPT-4 Technical Report (2024)
- Pacheco C, Ernst MD (2007) Randoop: feedback-directed random testing for java. Companion to the 22nd ACM SIGPLAN conference on object-oriented programming systems and applications companion. *OOPSLA '07*. Association for Computing Machinery, New York, pp 815–816
- Papineni K, Roukos S, Ward T, Zhu W-J (2002) Bleu: a method for automatic evaluation of machine translation. *Proceedings of the 40th annual meeting on association for computational linguistics. ACL '02*. Association for Computational Linguistics, USA, pp 311–318
- Putra IS, Rukmono SA, Perdana RS (2021) Abstract Syntax Tree (AST) and Control Flow Graph (CFG) Construction of Notasi Algoritmik. In: *2021 International conference on data and software engineering (ICoDSE)*, pp 1–6
- Ribeiro FGC, Pereira CE, Rettberg A, Santos Soares M (2018) Model-based requirements specification of real-time systems with UML, SysML and MARTE. *Softw Syst Model* 17:343–361
- Sahoo R, Nanda S, Mohapatra D, Patra M (2017) Model driven test case optimization of UML combinational diagrams using hybrid bee colony algorithm. *Int J Intell Syst Appl* 9:43–54
- Sawilowsky SS (2009) New effect size rules of thumb. *J Mod Appl Stat Methods* 8(2):597–599
- Schafer M, Nadi S, Eghbali A, Tip F (2024) An empirical evaluation of using large language models for automated unit test generation. *IEEE Trans Softw Eng* 50(01):85–105

- Takerngsaksiri W, Charakorn R, Tantithamthavorn C, Li Y-F (2024) TDD without tears: towards test case generation from requirements through deep reinforcement learning. [arXiv:https://arxiv.org/abs/2401.07576](https://arxiv.org/abs/2401.07576)
- Taromirad M, Runeson P (2023) A literature survey of assertions in software testing. Springer, Berlin, Heidelberg, pp 75–96
- Teixeira F, Silva G (2018) EasyTest: an approach for automatic test cases generation from UML activity diagrams, pp 411–417
- Zaharia M, Chowdhury M, Franklin MJ, Shenker S, Stoica I (2010) Spark: cluster computing with working sets. In: Proceedings of the 2nd USENIX conference on hot topics in cloud computing. HotCloud'10. USENIX Association, USA

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.